

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Análisis de Ejercicio 3

Estudiante:
Emily Lucila Menchú Menchú - 20472

Fecha: 25/08/2026

Análisis

Clases a definir:

- Main: punto de inicio del programa.

Modelo

- Orden: Clase que representa una orden de servicio.
- Taller: Clase que servirá para representar el taller y será el encargado de administrar la lista de órdenes.

Controlador

- ControladorTaller: Clase que servirá para controlar el flujo del programa y coordinar operaciones entre la vista y las clases del modelo.

Vista

- VistaTaller: se encarga solamente de pedir información y mostrar información al usuario.

Relaciones entre clases:

- Main tendrá una relación de dependencia con ControladorTaller, ya que solo lo utilizará para iniciar el programa de administración de ordenes.
- ControladorTaller y Taller tendrán una relación de agregación ya que el controlador tendrá una referencia al Taller pero Taller puede existir independientemente del controlador.
- ControladorTaller y VistaTaller tendrán una relación de agregación. El controlador también guardará una referencia de la Vista para poder pedir y mostrar información.
- Taller y Orden tendrán una relación de agregación. El taller administrará una lista dinámica de órdenes usando ArrayList<Orden>.

Preguntas:

- **¿Qué información deberá almacenarse dentro de la colección dinámica?**
Se almacenarán las órdenes de servicio, en este caso será una lista dinámica de objetos Orden.
- **¿Cómo identificará una orden específica dentro de la colección?**
Se identificará mediante el número de orden que es único. Para localizar se recorrerá la lista órdenes utilizando un ciclo for-each. En cada iteración se comparará el número ingresado con el número de la orden hasta encontrar una coincidencia.
- **¿Cómo agregará y eliminará órdenes de la colección dinámica?**
Para agregar primero se harán validaciones de los datos y se usará el método add() del ArrayList. Para eliminar primero se tiene que localizar y luego usar el método remove() enviando el objeto Orden.
- **¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?**
Se creará una variable temporal dentro del método correspondiente que será una lista dinámica, luego con un ciclo for-each se recorrerá la lista de ordenes comparando las placas de los autos y cuando haya coincidencias se agregarán esas órdenes a la lista temporal.

- **¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?**

Con un for-each se recorrerá la lista de órdenes y los costos se irán sumando a una variable total. En el caso del promedio se necesitará saber cuantas órdenes hay entonces se usará el método size() que tienen las listas dinámicas.

- **¿Cómo determinará cuál es la orden con el costo estimado más alto?**

Primero verificar si existen órdenes con isEmpty() y luego crear una variable de tipo Orden que almacenará el de mayor costo, luego se recorrerá la lista con un for-each comparando los valores de costo.

- **¿Qué situaciones dentro del programa pueden producir una excepción?**

Las principales situaciones son en la Vista ya que se pide un tipo específico de dato al usuario y si no ingresa el correcto lanza una excepción. También a la hora de buscar, modificar o cancelar órdenes pueden aparecer excepciones por busca, modificar o eliminar una orden que no existe.

- **¿Qué operaciones serán protegidas utilizando try-catch?**

Será usado en los métodos donde se le pide al usuario ingresar datos numéricos, también a la hora de buscar, modificar y eliminar órdenes porque puede que no existan.

- **¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?**

Se usará el finally para indicarle al usuario que la operación que se está ejecutando finalizó ya sea que haya sido exitoso o haya habido una excepción. Es importante que se ejecute independientemente de si falló o no para que el usuario esté informado. Por ejemplo en una búsqueda, ya sea que se haya encontrado o no la orden, finally siempre imprimirá el mensaje "Búsqueda finalizada".

Tabla Atributos

#	Tipo de dato	Nombre	Visibilidad	Valor inicial	Motivo	Clase
1	int	numeroOrden	private	Por constructor se enviará el dato ingresado por el usuar	Guardar el número de orden que servirá como su identificador único	Orden
2	String	nombrePropietario	private	Mediante constructor	Guardar el nombre del dueño del vehículo	Orden
3	String	placa	private	Mediante constructor	Guardar la placa del vehículo de la orden	Orden
4	String	descripcion	private	Mediante constructor	Guardar la descripción del servicio pedido	Orden
5	double	costoEstimado	private	Mediante constructor	Guardar el costo estimado del servicio	Orden
6	ArrayList<Orden>	ordenes	private	new ArrayList<>()	Guardar dinámicamente las órdenes que entran al taller sin determinar un número fijo de órdenes	Taller

7	Taller	taller	private	new Taller()	Mantiene el taller sobre el cual el controlador coordina las operaciones del sistema.	ControladorTaller
8	VistaTaller	vista	private	new VistaTaller()	Mantiene la vista utilizada por el controlador para solicitar y mostrar información.	ControladorTaller
9	Scanner	sc	private	new Scanner(System.in)	Permite a la vista recibir las entradas ingresadas por el usuario.	VistaTaller

Tabla de métodos

#	Retorno	Nombre	Visibilidad	Parámetros	Motivo	Clase
1	-	Orden	public	numeroOrden:int, nombrePropietario:String, placa:String, descripcion:String, costoEstimado:double	Método constructor de Orden que asigna los valores de sus atributos	Orden
2	int	getNumeroOrden	public	-	Obtiene el número que identifica la orden	Orden

3	String	getPlaca	public	-	Obtiene la placa para búsquedas de órdenes del mismo vehículo.	Orden
4	void	setDescripcion	public	descripcion:String	Modifica la descripción del servicio	Orden
5	void	setCostoEstimado	public	costoEstimado:double	Modifica el costo estimado	Orden
6	double	getCostoEstimado	public	-	Obtiene el costo estimado del servicio	Orden
7	String	toString	public	-	Obtiene un String con la información de la orden	Orden
8	-	Taller	public	-	Constructor de la clase Taller	Taller
9	void	agregarOrden	public	orden:Orden	Agrega una orden a la lista dinámica	Taller
10	void	modificarOrden	public	numeroOrden: int, nuevaDescripción: String, nuevoCosto: float	Para modificar una orden	Taller

11	Orden	buscarOrden	public	numeroOrden:int	Obtiene una orden por su número	Taller
12	boolean	eliminarOrden	public	numeroOrden:int	Busca y elimina una orden de la lista	Taller
13	ArrayList<Orden>	buscarOrdenesPorPlaca	public	placa:String	Busca en toda la lista y recolecta las órdenes asociadas al mismo automóvil	Taller
14	double	calcularCostoTotal	public	-	Suma los costos de todas las ordenes	Taller
15	double	calcularCostoPromedio	public	-	Calcula el promedio de los costos de órdenes	Taller
16	Orden	obtenerOrdenMayorCosto	public	-	Recorre la lista de órdenes y obtiene el de valor más alto	Taller
17	int	obtenerCantidadOrdenes	public	-	Obtiene la cantidad de órdenes almacenadas	Taller

18	ArrayList<Orden>	obtenerOrdenes	public	-	Obtiene la lista de ordenes para consulta	Taller
19	boolean	existeOrden	public	numeroOrden: int	Verifica si ya existe una orden con el mismo número	Taller
20	-	ControladorTaller	public	-	Constructor del controlador. Inicializa la vista y Taller	Controlador Taller
21	void	iniciar	public	-	Inicia el ciclo del programa	Controlador Taller
22	void	registrarOrden	private	-	Agrega orden obteniendo los datos ingresados por el usuario en vista.	Controlador Taller

23	void	consultarOrdenes	private	-	Solicita a Taller las órdenes y se las pasa a la vista para que las muestre	Controlador Taller
24	void	buscarOrden	private	-	Obtiene el numero de orden a través de la vista y se lo pasa a Taller para que busque la orden correspondiente para poder mostrárselo al usuario.	Controlador Taller
25	void	modificarOrden	private	-	Maneja la búsqueda y la modificación de una orden	Controlador Taller
26	void	cancelarOrden	private	-	Coordina la búsqueda y eliminación de una orden	Controlador Taller
27	void	consultarPorPlaca	private	-	Le pide a Taller la lista de órdenes de un automovil para mostrárselo al usuario a través de vista	Controlador Taller

28	void	mostrarReporteCostos	private	-	Le pide al taller el costo total y promedio y los envía a la vista	Controlador Taller
29	void	mostrarOrdenMayorCosto	private	-	Solicita al taller la orden con mayor y se lo envía a vista	Controlador Taller
30	void	mostrarCantidadOrdenes	private	-	Solicita la cantidad de órdenes y la envía a la vista	Controlador Taller
31	-	VistaTaller	public	-	Constructor de la vista	VistaTaller
32	int	mostrarMenu	public	-	Muestra el menú y retorna la opción ingresada	VistaTaller
33	String	pedirTexto	public	mensaje:String	Pide un al usuario que ingrese un texto	VistaTaller
34	int	pedirEntero	public	mensaje:String	Solicita al usuario un valor entero.	VistaTaller

35	double	pedirDecimal	public	mensaje:String	Solicita al usuario un valor decimal.	VistaTaller
36	void	mostrarMensaje	public	mensaje:String	Muestra mensaje al usuario	VistaTaller
37	void	main	public static	args:String[]	Punto de inicio del programa y crea a controlador	Main

Diagrama UML

Ejercicio 3 - Sistema de Control de Órdenes de Servicio de un Taller Automotriz
Diagrama de Clases UML
Emily Lucila Menchú Menchú - 20472

